CS 188 — Complete Study Guide

A reference document, organized by topic. Built from your notes, your midterm cheatsheet, and gap analysis against three past finals.

1. Search

State space = set of all possible configurations. **Minimal state space representation** = the most compact encoding that still distinguishes all relevant states (don't include info that doesn't affect the future).

Size of state space (rules of thumb):

- One object on an $M \times N$ grid: MN
- k independent objects on $M \times N$ grid: $(MN)^k$
- Toggle grid (each cell on/off): 2^{MN}

Max branching factor:

- One object, 4 directions (L/R/U/D): 4
- k independent objects each with 4 moves: 4^k

Uninformed search algorithms:

Algorithm	Frontier structure	Behavior
DFS	Stack (LIFO)	Goes to the end of one path before backtracking. Not optimal, not complete on infinite/looping spaces unless graph search used.
BFS	Queue (FIFO)	Explores ring by ring (like ripples in a pond). Optimal **only** if all step costs equal (finds shortest path in # of edges, not cost).
UCS	Priority queue by cumulative cost	Push (node, cumulative cost). Expand/pop/test the lowest-cumulative-cost node. **Terminate when goal is popped**, not when first generated. Optimal for any non-negative costs.

Tree search vs. graph search: tree search can revisit states (infinite loops possible); graph search keeps a *closed/visited set* and never re-expands a state. Graph search is what guarantees termination on graphs with cycles.

Depth-limited / Iterative Deepening:

- Depth-limited search: cutoff at depth L, treat as failure past that (not necessarily "no solution" — could just be too deep).
- Iterative deepening (IDS): repeat DFS with limit L = 1, 2, 3, ... stop at first solution. Combines DFS's space efficiency with BFS's completeness/optimality (for uniform step cost).
- IDA*: same idea but the "limit" is on f = g + h, not depth. Each pass, limit = f(start); each failed pass, raise limit to the smallest f-value that got rejected as the new limit; repeat. Optimal, uses much less memory than A*.

Admissible heuristics: $0 \leq h(n) \leq h^*(n)$ — never overestimates true remaining cost to goal.

- If admissible and used in A* **tree search** $\rightarrow$ optimal.
- If admissible but not consistent, A* **graph search** may not be optimal (graph search needs consistency for the optimality guarantee — know both terms).
- **Consistent** (monotonic): $h(n) - h(n') \leq \text{cost}(n, n')$ for every edge $n \rightarrow n'$. Consistency $\Rightarrow$ admissibility (not vice versa).
- If h is admissible, the true optimal cost is $C = h^*(\text{start})$. If h is not admissible, A*'s output cost may exceed the true optimum (it may find a suboptimal path).
- **Combining heuristics**: $\max(h_1, h_2)$ is always admissible if h_1, h_2 are. $\min(h_1, h_2)$ is also always admissible (weaker/less informed, but never overestimates — if both stay $\leq h^*$, their min does too).

- A heuristic derived from a **relaxed problem** (fewer constraints, e.g., ignore walls, allow overlap) is generally admissible because relaxing can only lower the true cost, never raise it above the real h^*.

Greedy search: expands node with lowest h(n) only (ignores path cost so far). Not optimal, not complete in general — behaves like DFS in the worst case.

Beam search: like BFS/greedy but only keeps the top-k nodes (by heuristic/value) at each level, discards the rest. **Not optimal and not complete** in general — a wider beam approaches brute force / optimality, narrower beam is closer to greedy behavior. It is a memory-limited compromise, not a guaranteed algorithm.

A* = UCS + heuristic: expand node minimizing $f(n) = g(n) + h(n)$.

Watch-out / always-sometimes-never traps:

- BFS tree search vs. BFS graph search can return paths of **different length** in general (tree search may revisit and find a longer path where graph search finds the true shortest) — don't assume they always agree.
- Alpha-beta pruning (see Games) never changes the *value* or *optimal action* versus unpruned minimax — it's purely an optimization, regardless of tree shape/balance.

2. CSPs (Constraint Satisfaction Problems)

Setup: variables, domains, constraints. **Constraint graph**: edge between variables sharing a constraint. Unary constraint = restricts one variable's own domain (apply immediately, before search). Binary constraint = between two variables. Higher-order constraints also possible.

Backtracking search: DFS where at each step you assign one variable a value from its domain, checking constraints as you go; if a constraint is violated, backtrack. Worst case $O(d^n)$ for n variables, d values each.

Ordering heuristics (only help average-case runtime — do NOT change worst-case complexity or guarantee less backtracking in the worst case):

- **MRV (Minimum Remaining Values)**: pick the unassigned variable with the *smallest* domain left. Tie-break: variable involved in the most constraints with other unassigned variables (most constraining).
- **LCV (Least Constraining Value)**: for the *value* of the variable you're about to assign, pick the value that rules out the fewest values in neighboring variables' domains. (Simulate assigning it, count how much it shrinks neighbors' domains, pick the value that shrinks least.)

Filtering — enforcing arc consistency:

- Arc $X \to Y$ is consistent if: for every value in X's domain, there's *some* value in Y's domain satisfying the constraint between X and Y. If not, remove the offending value(s) from X.
- **Forward checking**: cheap, lazy, 1-step lookahead. When you assign a variable, only shrink the domains of its *immediate* neighbors. Ignores neighbors-of-neighbors.
- **AC-3 (full arc consistency)**: initialize the queue with **all arcs**. Enforce each arc; if a domain shrinks, **re-enqueue all arcs pointing INTO that variable** that aren't already queued (because that variable's change might break consistency for its own neighbors' arcs). Continue until queue empty. Leaves you with much smaller domains across the board vs. forward checking, at the cost of more computation. AC-3 worst case: $O(n^2 d^3)$ (n^2 arcs at most, each re-enforced up to d times, each enforcement checks d^2 pairs).
- If any domain reaches zero values $\to$ backtrack (dead end).
- Enforcing full arc consistency **before search starts** can prune the space, but does NOT guarantee a solution or no-solution — you may still need to search (or discover unsolvability) after.

Structure-based speedups:

- **Tree-structured CSPs**: if the constraint graph is a tree (no cycles), can be solved with **no backtracking** — topologically sort the graph (pick a root), enforce arc consistency child $\to$ parent in reverse order (a backward "directed arc consistency" pass), then assign each variable greedily in forward topological order (parent before child) — guaranteed consistent given no cycles.

- **Cutset conditioning**: pick a small set of variables (the "cutset") whose removal makes the remaining graph tree-structured. Enumerate every possible assignment to the cutset, and for each, solve the resulting tree-structured **residual CSP** in polynomial time. Total residual CSPs = product of cutset variables' domain sizes. **Important**: a solution to a residual CSP isn't guaranteed to solve the *original* CSP — the cutset assignment might violate a constraint that exists only between cutset variables themselves, so you must check.
- **Min-conflicts** (local search, not backtracking): start with every variable assigned some (possibly invalid) value, ignoring constraints. Repeat: randomly select a variable currently violating some constraint, reassign it to the value that minimizes # of constraint violations (test each possible value). Repeat until no violations or give up. No completeness guarantee (can loop/plateau); fast in practice for large CSPs (e.g., n-queens).

Independent sub-CSPs: if the constraint graph splits into disconnected components, solve each independently — the full solution set is the **Cartesian product** of each component's solution sets.

Watch-outs:

- Backtracking search returning a value $\neq$ that value being *guaranteed* correct in the "no-hidden-constraint" sense — a solution to a partial/relaxed model is only a valid real-world answer if it's consistent in **every** solution to the CSP, not just the one backtracking happened to find.
- MRV and LCV help average performance but never *guarantee* reduced backtracking or change worst-case bounds.

3. Adversarial Search / Games

Minimax: bottom-up from leaves. At a MIN node, take the min of children's values; at a MAX node, take the max. The root's value is the game's minimax value (assuming optimal play from both sides).

Expectimax: like minimax, but chance nodes (circles) take the **expected value** (probability-weighted average) of children instead of min or max. Used when an opponent/environment is random or suboptimal rather than adversarial-optimal.

Alpha-beta pruning:

- α = best value MAX can guarantee so far on the path to root (starts at $-\infty$). β = best value MIN can guarantee so far (starts at $+\infty$).
- Pass current α, β down to children unchanged when first descending.
- At a leaf: return its value directly to parent.
- At a MIN node: for each child returning v, update $\text{value} = \min(\text{value}, v)$, then $\beta = \min(\beta, \text{value})$. If $\alpha \geq \beta$: **prune** — stop, skip all remaining children, return current value.
- At a MAX node: for each child returning v, update $\text{value} = \max(\text{value}, v)$, then $\alpha = \max(\alpha, \text{value})$. If $\alpha \geq \beta$: prune remaining children, return current value.
- After a node finishes (all children checked or pruned early), go to parent's **next unexplored child**.
- **Alpha-beta pruning never changes the root value or the optimal root action** vs. unpruned minimax — regardless of tree shape (balanced or unbalanced), it's a pure optimization.
- **Node visitation rule**: with correct alpha-beta, you cannot prune the *first* child of any node (nothing to compare against yet), and you cannot leave an unpruned child under an already-pruned parent (if the parent is pruned, none of its remaining children/subtree is visited).
- Pruning decisions can depend on tie-breaking convention: whether you prune "on equality" ($\alpha \geq \beta$ prunes immediately at equality) needs to be stated/assumed consistently in a problem.

Depth-limited minimax + evaluation functions: cut the tree early at a fixed depth and plug in an **evaluation function** (heuristic estimate of a state's value) instead of the true minimax value. Not guaranteed optimal (the eval function is only an approximation). Which depth is reached from a given node can change unpredictably as depth limits change (a shallow search can flip actions vs. a deeper one, even non-monotonically) — deeper is not strictly guaranteed to be "better" locally at every node, only in aggregate.

Multi-agent games (non zero-sum) / games with >2 players: each node type maximizes/minimizes a *different index* of a utility **tuple** (e.g., each player has their own value, and a "MAX-for-player-i" node updates only that player's coordinate while just passing through the others from the chosen child). **Cannot prune in general multi-agent games** — because each player's

utility component is independent of the others, you cannot bound one player's outcome using knowledge of another's, so the alpha-beta "if $\alpha \geq \beta$ skip" logic doesn't hold across mismatched objectives.

Suboptimal opponents: if you know an opponent is suboptimal (e.g., won't always take the "best" move for them), you cannot always safely prune the same way as with a known-optimal opponent — pruning that assumes optimal adversarial play can be invalid against a suboptimal one, since a "worse" branch for them might still be chosen.

Bound/inequality-style questions (general method): when asked "for what range of a parameter (ϵ, discount γ, threshold x) does action/value A become preferred over B," set the two expressions for A and B **equal**, solve for the parameter — that gives the boundary; then reason about which side of the boundary makes A $\geq$ B (or >) based on the direction the parameter pushes the expression. Always double check strict vs. non-strict inequality based on how ties are defined in the problem (e.g., "Tie $\rightarrow$ prefer left" changes whether the boundary itself is included).

4. MDPs (Markov Decision Processes)

Definition: $(S, A, T, R, \gamma, s_0)$ = states, actions, transition function, reward function, discount factor, start state.

- $T(s, a, s')$ = probability of landing in s' after taking action a in state s.
- $R(s, a, s')$ = immediate reward for that transition.
- **Utility of a sequence**: $U = r0 + \gamma r1 + \gamma^2 r2 + ...$ where $rt = R(st, at, st{+}1)$.
- **Policy** $\pi(s)$ = the action to take in state s.

Bellman equations:

- $Q(s, a) = \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V(s')]$ — expected value of taking action a in state s.
- $V^*(s) = \max_a Q^*(s, a) = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V^*(s')]$ — optimal value taking the best action.
- **Value iteration** update: $V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V_k(s')]$.
- $Q_{k+1}(s, a) = \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma \max_{a'} Q_k(s', a')]$.
- If policy π is fixed (**policy evaluation**, not optimization): drop the max, plug $\pi(s)$ into the formula: $V^{\pi}_{k+1}(s) = \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V^{\pi}_k(s')]$. Only the state where the action *originates* has its value used/updated in a single-sample TD-style version — successor's value isn't touched by this update.

Convergence facts:

- If $0 < \gamma < 1$, value iteration is guaranteed to converge to the same fixed values **regardless of initial values** (and MDP guarantees termination for finite MDPs).
- **Iterations needed to converge to true optimal value** = exactly the number of edges in the **longest path** to a terminal/goal state (worst case, reward propagates one edge per iteration).
- With $V_0 = 0$ everywhere, V_k first becomes nonzero for a state once a path of length k from that state reaches a rewarding transition.
- **Policy converges strictly before values do** (or at best at the same iteration) — never the reverse. Once policy stops changing between iterations, it has found the optimal policy even if the numeric values are still shifting slightly.
- A **cycle with positive reward** breaks the "converges in finitely many iterations" guarantee in general — deterministic transitions + all-zero rewards except one strictly positive terminal reward is a specific edge case that still allows convergence; but generally reward cycles need care.
- **Lower discount factor** $\rightarrow$ future rewards decay faster $\rightarrow$ agent favors smaller near-term rewards over larger long-term ones (short-term-biased). γ close to 1 $\rightarrow$ long-term-focused, patient agent.
- Changing γ **can change the optimal policy**, not just the numeric values.

Q-values vs. V-values: $V^*(s) = \max_a Q^*(s, a)$. Optimal action at s = $\arg\max_a Q^*(s, a)$.

Complexity: policy iteration's policy evaluation step is $O(|S|^3)$ if solved exactly via linear equations (matrix inversion), or iteratively (cheaper per round but many rounds). Policy iteration is guaranteed to converge to the optimal policy in a **finite** number of iterations (finite number of possible policies, and each iteration's policy-improvement step never makes the policy worse — $V^{\pi'} \geq V^{\pi}$ for all states, not necessarily strictly greater).

Bound-derivation pattern for MDPs: e.g., "what is the minimum k for values to converge" or "for what γ does policy X become at least as good as policy Y" — set up the two value expressions, solve the equality case, then check the direction of the inequality based on which term grows/shrinks with the parameter. This is the same general method as the Games bound section above — practice keeping every step's arithmetic written out explicitly rather than combining steps mentally, since multi-step V-iteration tracking is an easy place to make small arithmetic slips.

Decision-network reformulation of an MDP: an MDP can be redrawn as a decision network where the action node influences a chance node (whether the action "succeeds," e.g.), which together determine the reward/utility node. Rule: **action nodes should not point directly into chance nodes that represent probabilistic environment behavior in a way that bypasses the actual state transition** — the structure must reflect that the utility depends on both the action taken and the (possibly probabilistic) outcome of attempting it, not have the action deterministically "become" the outcome.

5. Reinforcement Learning

Offline vs. online:

- **Offline planning** (value/policy iteration): T and R are *known*; solve by "staring at the map."
- **Online (RL)**: T and R **unknown**; must act and learn from (s, a, s', r) samples.

Model-based RL: learn estimates $\hat{T}$, $\hat{R}$ from experience, then run value/policy iteration on the learned model.

- $\hat{T}(s, a, s') = \text{Count}(s, a, s')/\text{Count}(s, a)$
- $\hat{R}(s, a, s') = \text{average of observed rewards for that } (s, a, s')$

Model-free RL: learn V or Q directly from samples, without ever estimating T or R.

Direct evaluation: given a *fixed* policy π, follow it repeatedly, and for each state log the total observed discounted return from that visit onward; $\hat{V}(s) = \text{average of all observed returns starting at } s$. No T/R needed, but wasteful (ignores the Markov structure — doesn't use info from *future* visits to inform earlier states, and needs full episodes).

Temporal Difference (TD) Learning: updates a *fixed policy's* value estimate online using single transitions. $V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha[R(s, \pi(s), s') + \gamma V^\pi(s')]$ Only the state where the transition **originated** gets updated (not s'). This is *passive* — it evaluates a fixed π, does not itself improve the policy.

Q-learning (model-free, off-policy — converges to optimal Q^* regardless of the behavior policy used to generate samples): $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$

- **Convergence conditions**: every (s,a) pair must be visited **infinitely often**, and α must be decreased to 0 over time (not fixed — with α = 1 fixed, updates always fully overwrite instead of averaging in new information, which can prevent stable convergence). Some mix of exploration and exploitation is required — pure exploitation (e.g., "always take the currently-best action") risks never trying some (s,a) pairs infinitely often.

Epsilon-greedy exploration: with probability ϵ, act uniformly at random over all K available actions; with probability $1 - \epsilon$, take the current best action.

- $P(\text{any specific non-best action}) = \epsilon/K$
- $P(\text{best action}) = (1 - \epsilon) + \epsilon/K$

Exploration functions (alternative to plain ϵ-greedy — modifies the Q-update itself): $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} f(Q(s', a'), N(s', a'))]$ where $f(u, n)$ = exploration function combining the value estimate u and visit count n. A good exploration function should **encourage trying less-visited actions** — i.e., f should be *larger* for smaller N (so unvisited/rare actions look artificially more attractive early on), and converge back to just u as N grows large (so eventually behaves like pure exploitation). Typical shapes: $u + k/(N + 1)$ or $u + c/\sqrt{N}$ — decreasing bonus in N.

Feature-based (Approximate) Q-learning: $Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \ldots$

- **Sample** $= r + \gamma \max_{a'} Q(s', a')$ (immediate reward + best possible future score).
- **Difference (TD error)** $= \text{sample} - Q(s, a)$ = reality minus expectation.

- **Weight update**: $w_i \leftarrow w_i + \alpha \cdot \text{difference} \cdot f_i(s,a)$ for each feature i.
- Generalizes across unseen states that share feature values with seen states — this is the key advantage over tabular Q-learning (tabular Q-learning stores one entry per (s,a) pair, $O(|S||A|)$ parameters, cannot generalize to unseen state-action pairs at all).
- **State-only features** (features that don't depend on the action) → Q-value differences across actions in the same state come entirely from the weights tied to action-dependent features; if all features are state-only, the model cannot differentiate one action from another in the same state (same predicted Q regardless of a).

Which method needs a model, and which generalizes:

Method	Needs T, R?	Generalizes to unseen (s,a)?	Learns fixed π or optimal π^*?
Value/Policy Iteration	Yes	N/A (planning, not learning from samples)	Optimal
Direct Evaluation	No	No	Fixed π only
TD Learning $V^\pi(s)$	No	No	Fixed π only
Tabular Q-learning	No	**No**	Optimal (Q^*)
Approximate/feature Q-learning	No	**Yes**	Optimal (approx.)
Neural network Q-function	No	**Yes** (best generalization, nonlinear)	Optimal (approx.)

Neural network Q-functions: same idea as feature-based Q-learning but with a deep net instead of a linear combination — same loss-minimization pattern: $\text{Loss} = [(r + \gamma \max_{a'} \hat{Q}(s', a')) - \hat{Q}(s,a)]^2$, minimizing this pushes predicted Q-values to better satisfy the Bellman equation (not to directly minimize/maximize Q universally, nor to approximate just the immediate reward).

- **Generalization failure mode**: if the network's *input* doesn't encode enough information to distinguish two different underlying tasks/environments that produce the same low-level state (e.g., same local grid pattern but different maps), the learned Q-function isn't really "how to act well in general" — it's tied to the specifics it was trained on. Fix: use an input representation with enough distinguishing information (richer or task-identifying features); more/longer training on the same limited representation doesn't add the missing information.

General generalization-vs-overfitting facts (applies across RL/ML broadly):

- Running an algorithm **longer** → training error tends to decrease (or stay same).
- Removing features → error stays the same or increases (fewer signals available).
- Training on **more data alone** does not guarantee improvement if the input representation itself is insufficient — but with a good representation, more data generally helps generalization.
- Adding features (even irrelevant/non-linear ones) → training error decreases or stays the same (never increases, more capacity).

6. Probability Foundations

- $P(A|B) = P(A,B)/P(B) = [P(B|A) \cdot P(A)]/P(B)$ (Bayes' Rule)
- $P(A \cap B) = P(A) \cdot P(B|A)$
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$; if mutually exclusive, $P(A \cap B) = 0$ so it's just the sum.
- **Chain rule**: $P(X,Y) = P(X) \cdot P(Y|X)$. $P(X,Y,Z) = P(X) \cdot P(Y|X) \cdot P(Z|X,Y)$ — if Z $\perp\!\!\!\perp$ X given Y, the last term simplifies to $P(Z|Y)$.
- **Independence**: X $\perp\!\!\!\perp$ Y iff $P(X,Y) = P(X)P(Y)$ for **all** values of X and Y (not just some).
- **Conditional independence**: X $\perp\!\!\!\perp$ Y | Z iff $P(X,Y|Z) = P(X|Z) \cdot P(Y|Z)$.
- **Law of Total Probability**: $P(A) = \sum_i P(A|B_i) \cdot P(B_i)$ for a partition {B_i}.

- Bayes' Rule with extra conditioning: $P(A|B,C) = P(B|A,C) \cdot P(A|C)/P(B|C)$, or treating (B,C) as one joint condition: $P(A|B,C) = P(B,C|A) \cdot P(A)/P(B,C)$.

7. Bayes Nets — Structure & Independence

Joint distribution factorization: the joint probability over all variables in a Bayes Net = the **product of each node's CPT given its parents**: $P(X_1, ..., X_n) = \Pi_i P(X_i|Parents(X_i))$.

Number of CPT entries for a node with domain size d and k parents each with domain size d: $d^{k+1} - d^k$ entries (subtract the ones determined by summing to 1); a root node (no parents) has $d - 1$ free entries.

Independence with no edges: if a Bayes Net has **no edges**, all variables are mutually independent: $P(X|anything) = P(X)$. This means: to compute $P(query|evidence)$, you don't need any join/eliminate operations if query and evidence are disjoint single variables — just return the relevant CPT directly. If there are **multiple query variables** (e.g., $P(X, Y|Z)$ with no edges), you still need to **join** $P(X)$ and $P(Y)$ together since they're separate CPTs, even though they're independent.

D-separation (determining conditional independence directly from graph structure): for two variables to be independent given an evidence set Z, **every undirected path** between them must be "blocked." A path through an intermediate triple is blocked at that node depending on the triple type:

- **Causal chain** $A \rightarrow B \rightarrow C$: active (unblocked) if B is **not** in evidence; blocked if B **is** given.
- **Common cause** $A \leftarrow B \rightarrow C$: active if B is **not** given; blocked if B **is** given.
- **Common effect (collider)** $A \rightarrow B \leftarrow C$: the reverse of the other two — active **only if** B (or any descendant of B) **is** given; blocked if B and all its descendants are **not** given.
- A path is active only if **every** triple along it is active; if even one triple is blocked, that whole path is blocked. Independence holds only if **all paths** between the two variables are blocked.

⚠ **Explaining away (this is a common trap — read carefully)**: this is the collider case. Two variables that are marginally *independent* (e.g., two unrelated causes of a shared effect) become **dependent once you condition on their common effect** (or a descendant of it). Intuition: if you know the effect happened, and you learn one cause is now less/more likely, that changes your belief about the other cause too ("explains away" the effect without needing the other cause). **Rule of thumb to avoid the midterm-style mistake**: whenever a common-effect node (or its descendant) is in the evidence set, do NOT assume its parent causes stay independent — check explicitly, they generally become dependent. This applies even if there's no direct edge between the two parent variables at all.

Markov blanket: for a node X, its Markov blanket = its parents, its children, and its children's *other* parents (co-parents). Given its Markov blanket, X is conditionally independent of every other variable in the network. This is exactly the set of variables whose CPTs you need to join when **resampling X in Gibbs sampling** (see below).

8. Inference: Enumeration & Variable Elimination

Inference by enumeration: to compute $P(query|evidence)$:

1. Write the full joint as the product of every CPT (with evidence values plugged in where applicable).
2. Sum out ("marginalize") every **hidden** variable (not query, not evidence) from that product to get the numerator = $P(query, evidence)$.
3. Do the same summing every non-evidence variable (including query) to get the denominator = $P(evidence)$.
4. Normalize: numerator / denominator (equivalently, just compute the numerator for each value of the query variable and normalize those against each other — faster).

Downside: this builds one giant joint table over all variables before summing — expensive.

Variable elimination (more efficient — never materializes the full joint):

1. Write down all the initial CPTs (factors), with evidence values plugged in.
2. Eliminate hidden variables **one at a time** (not query/evidence variables): for the variable being eliminated, **join** (multiply) together every factor that mentions it into one combined factor, then **sum out** that variable from the combined factor,

producing a new factor over the remaining variables.
3. Repeat for each hidden variable in the chosen elimination order.
4. Multiply remaining factors (which now only involve query + evidence), normalize.

- **Order matters for efficiency** (not correctness) — every valid order produces the same final answer, but different orders create differently-sized intermediate factors along the way. **Factor size** = product of the domain sizes of the *unobserved* (not-yet-summed, not-evidence) variables remaining in that factor. Picking an order that keeps intermediate factors small (e.g., eliminating variables that appear in fewer remaining factors first, or that "disconnect" the graph quickly) is more efficient — there's no single universal best order, it depends on graph structure.

9. Sampling Methods

All sampling methods approximate $P(query|evidence)$ using generated samples instead of exact inference. Samples must always be generated in an order consistent with the Bayes Net's **topological order** (parents before children) — this applies to every method below.

Prior Sampling: sample every variable normally, straight from its CPT given its already-sampled parents (no evidence used during generation at all). Then, to estimate any query, just count frequencies among the generated samples matching what you want. **Wasteful**: most generated samples may not even match the evidence you care about, and evidence is thrown away/checked only after the fact.

Rejection Sampling: same as prior sampling, but **immediately discard (reject)** any sample as soon as it's generated with a value inconsistent with the evidence (don't wait until the full sample is built if you can check earlier). Still wasteful — rejects a lot of samples, especially when evidence is unlikely a priori. Only prior and rejection sampling can produce samples that are then thrown out; likelihood weighting and Gibbs never generate a sample inconsistent with evidence, so they're not "rejection"-style methods.

- **Estimating a conditional probability from surviving samples**: count how many of the *non-rejected* samples match the full query+evidence combination, divide by total non-rejected samples (not the original total generated).
- More evidence variables / less likely evidence $\rightarrow$ **more samples rejected** in general.

Likelihood Weighting: fixes rejection sampling's waste by **never sampling evidence variables randomly** — instead, force each evidence variable to its given observed value, and **skip the random draw** for those (no rejection ever needed). Every other (non-evidence) variable is still sampled normally from its CPT given already-assigned parents.

- Every sample gets a **weight** = the product, over all evidence variables, of $P(\text{observed value} \mid \text{its parents' values in this sample})$.
- **Formally**: $\text{WEIGHT}(\text{sample}) = \prod_{ei \in \text{evidence}} P(ei \mid \text{Parents}(ei))$ (parents' values as sampled/fixed in that particular sample).
- To estimate $P(query|evidence)$: sum the weights of all samples matching each query value, then normalize those weighted sums against each other.
- Downside: with many evidence variables, especially "downstream"/unlikely ones, weights can become very small (though never zero/rejected) — sample quality degrades since evidence doesn't influence the *upstream* variables' sampled values at all, only the final weight.

Gibbs Sampling (an MCMC method): maintain **one full current sample** (assignment to every variable, consistent with evidence held fixed throughout — evidence variables are never resampled). Repeatedly:

1. Pick one non-evidence variable X to **resample**.
2. Compute the distribution of X given its **Markov blanket** (all its parents, children, and children's other co-parents) fixed at their current values in the sample — this requires **joining exactly the CPTs that mention X**: $P(X)$ itself (if root) or $P(X|parents)$, plus every child's CPT $P(child|X, child's - other - parents)$.
3. Sample a new value for X from that resulting distribution; update the current sample.
4. Repeat, cycling through non-evidence variables, for many iterations.

- Key distinguishing fact: unlike the other 3 methods, Gibbs **only ever changes one variable at a time** per step, keeping everything else fixed — it never re-samples the entire assignment at once.

- The distribution needed to resample X can be written *equivalently* several ways as long as it conditions on X's full Markov blanket (or any superset of it, e.g., conditioning on more variables than the Markov blanket strictly requires still works and gives the same distribution, since the extra variables are already independent of X given the blanket).
- Because the network encodes exact conditional dependencies, some resulting full assignments are **impossible** (probability exactly 0, e.g., if a deterministic-like CPT entry has probability 0 for a certain combination) — Gibbs sampling will never resample a variable into a state making the sample's probability 0 under the CPTs.

10. HMMs & Particle Filtering

HMM structure: hidden state sequence X_t (Markov chain — each X_t depends only on X_{t-1}), with observations E_t depending only on X_t (**Sensor/Observation Model**), and X_t evolving via the **Transition/Dynamics Model** $P(X_t|X_{t-1})$.

Belief updates (exact inference):

- Initialize: $B(X_0) = P(X_0)$.
- **Time elapse (predict) update**: $B'(X_{t+1}) = \sum_{x_t} P(X_{t+1}|X_t = x_t) \cdot B(x_t)$.
- **Observation update**: after time elapse, incorporate the new evidence: $P(E_{t+1}, X_{t+1}|E_{1:t}) = P(E_{t+1}|X_{t+1}) \cdot B'(X_{t+1})$, then **normalize**: $B(X_{t+1}) = [P(E_{t+1}|X_{t+1}) \cdot B'(X_{t+1})] / \sum_{x_{t+1}} P(E_{t+1}|x_{t+1}) \cdot B'(x_{t+1})$.
- General pattern: **time elapse always uses the transition model only; observation update always uses the sensor model only**, and only the observation update step requires normalization (time elapse already preserves total probability = 1 automatically since it sums a valid distribution against valid transition probabilities).
- **Stationary distribution**: as t $\to \infty$, if the transition matrix has one, repeated time-elapse updates converge to a fixed distribution that doesn't change under another time-elapse step — solve $\pi = \pi \cdot T$ (the distribution that is a fixed point of the transition matrix) to find it.

Particle Filtering (approximate inference via samples, used when exact inference is too expensive — e.g., continuous or huge state spaces):

1. **Initialize**: N particles, each assigned a starting state (often uniformly, or drawn from the prior).
2. **Time elapse**: for each particle independently, sample its new state from $P(X_{t+1}|X_t = currentparticle's state)$ — this is *unweighted*, each particle just moves.
3. **Observation update (weighting)**: after new evidence e_{t+1} arrives, assign each particle a weight = $P(e_{t+1}|particle's current state)$ (using the sensor model). Particles are **not moved** in this step, only weighted.
4. **Resample**: draw N new particles **with replacement**, with probability proportional to each particle's weight (normalize weights first, or just use raw weights proportionally). This concentrates particles in the states currently best-supported by the evidence and discards unlikely ones.
5. Repeat steps 2-4 for each new time step.

Key particle filtering facts:

- Estimating a probability from particles: $P(\text{state}) \approx \frac{\#\text{ particles in that state}}{\text{total }\#\text{ particles}}$.
- Particle filtering is an **approximation** — even if *all* particles happen to be in one state at some point, that does not *guarantee* the true hidden state is actually that state (it's a belief estimate, not certainty).
- After a resampling step, if all particles land in the same state, that's a legitimate but not-guaranteed outcome — resampling with replacement genuinely can (and with skewed weights, often does) put many/all particles in the same place.
- Immediately after a **time-elapse update** (no observation/resampling yet), if all particles started in the same state, whether they end up in the same state again after transitioning depends entirely on whether the transition model is deterministic from that state — with a stochastic transition model, particles can (but aren't guaranteed to) spread to different successor states.
- All particles resulting from resampling that land in the exact same state will always be assigned the exact same weight in the *next* observation update (weight depends only on state + observation, not on particle identity/history).

11. Decision Networks & Value of Perfect Information (VPI)

Decision network: extends a Bayes Net with **action/decision nodes** (rectangles — the choices an agent controls) and a **utility node** (diamond — depends on relevant chance and action nodes). Chance nodes (ovals) behave as in a normal Bayes Net. Rule of thumb for validity: action nodes represent choices the agent makes; they should influence chance/utility nodes downstream, but a chance node representing a probabilistic real-world outcome should not be modeled as directly, deterministically *set* by an action node bypassing genuine uncertainty — and the utility node's parents should be exactly the variables that actually determine the payoff (both the action and the relevant outcome, if both matter).

Expected Utility of an action: $EU(\text{Action}) = \sum_{\text{outcomes}} P(\text{outcome}) \cdot U(\text{Action}, \text{outcome})$

Expected Utility given additional information: $EU(\text{Action} \mid \text{info}) = \sum_{\text{outcomes}} P(\text{outcome} \mid \text{info}) \cdot U(\text{Action}, \text{outcome})$

Maximum Expected Utility (MEU):

- Given some fixed info (or none): $MEU(\text{info}) = \max_{\text{Action}} EU(\text{Action} \mid \text{info})$ — the best you can do once you already know "info."
- **Expected value of knowing "info" before deciding** (i.e., value of the *random variable* representing what info you'd get, before you observe it): $MEU(\text{INFO}) = \sum_{\text{info}} P(\text{info}) \cdot MEU(\text{info})$ — average the best-achievable-utility over every possible value the info variable could take, weighted by how likely each value is.

Value of Perfect Information: $VPI(\text{INFO}) = MEU(\text{INFO}) - MEU(\text{no info})$

- **VPI is always** ≥ 0 — more information can never hurt expected utility (you can always ignore it and act as if you didn't have it).
- **VPI = 0** exactly when the information node is conditionally independent of every parent of the utility/decision node given what's already known — i.e., learning it wouldn't change your optimal action or its expected payoff.
- To compute VPI for an unknown-valued piece of information: compute $MEU(\text{info} = \text{each possible value})$ for every value the info could take, weight by $P(\text{info} = \text{that value})$, sum to get $MEU(\text{INFO})$; subtract the baseline $MEU(\emptyset)$ (best you can do with no extra info at all).
- **General "how much should you pay for information" pattern**: an agent should be willing to pay up to $VPI(\text{INFO})$ to acquire that information before acting — if the actual cost of acquiring the info exceeds VPI, it's not worth gathering.
- **Why repeatedly gathering the "same" information again doesn't help**: once you've already observed/incorporated a piece of information, learning it again gives you nothing new — VPI of already-known information is 0. If a model doesn't reflect this (e.g., no cost to "observe" and living reward is positive), an agent can pathologically loop trying to re-gather info; fixes include a negative cost per gathering action, or a discount factor less than 1 (making the agent prefer to stop stalling and act).

Sequential decision networks (belief-state MDP view): when decisions unfold over multiple time steps with actions, observations, and rewards each step, you can track a running **belief state** b_t summarizing everything relevant from the history so far (an MDP defined over belief states instead of raw hidden states). Key properties:

- The belief state is a sufficient statistic: knowing b_t alone is enough to act optimally going forward — you do **not** need to remember the raw history of past actions/observations, because the belief state already captures everything about the past that matters for predicting the future (a consequence of the Markov property applied at the belief-state level).
- VPI of a **future** observation, conditioned on already knowing the *state* that observation would reveal, is 0 (redundant information).
- $VPI(\text{current info}, \text{future info})$ can be **strictly greater than** $VPI(\text{current info})$ alone (more information sources generally help at least as much, sometimes strictly more) — but **never less** (monotonic non-decreasing in the amount of information considered).

12. Naive Bayes & Classification

Setup: predict a class C from a feature vector $(X_1, ..., X_n)$.

Naive Bayes independence assumption: every feature X_i is assumed **conditionally independent of every other feature** X_j**, given the class** C: $P(X_i, X_j|C) = P(X_i|C) \cdot P(X_j|C)$. This is a modeling *assumption*, not something guaranteed by real data — it's an approximation that keeps the model tractable.

Classifier formula: $P(C \mid X_1, ..., X_n) \propto P(C) \cdot \prod_i P(X_i \mid C)$ — proportional because the denominator $P(X_1, ..., X_n)$ is the same across all classes C, so for classification (picking the argmax class) you can skip normalizing and just compare the unnormalized products.

Required CPTs for a Naive Bayes Bayes-Net: $P(C)$ (the class prior) and $P(X_i|C)$ for every feature i. You do **not** need any $P(X_i|X_j)$ or joint feature CPTs — that's exactly what the independence assumption avoids needing.

Classification procedure: for a new data point, compute the unnormalized joint $P(C = c) \cdot \prod_i P(X_i = x_i|C = c)$ for **every possible class value** c, and predict the class with the **largest** value (argmax) — no need to normalize into true probabilities unless the exact probability is asked for.

Generalizing to "pick an action among scenarios": same structure — treat scenarios/evidence as independent given the action: $P(\text{scenario}_1, ..., \text{scenario}_k \mid \text{action}) = \prod_j P(\text{scenario}_j \mid \text{action})$; compute for every candidate action, pick the max.

Laplace Smoothing: adds a pseudocount k (strength) to every count before normalizing, e.g., $\hat{P}(x) = \dfrac{\text{count}(x) + k}{N + k \cdot |\text{values}(x)|}$.

- **Purpose**: prevents zero-probability estimates for value combinations that just didn't happen to appear in the (finite) training set — a raw MLE estimate of exactly 0 is often wrong and overconfident, especially with small data.
- **k = 0** reduces to plain MLE — best fit to *training* data, but prone to **overfitting** (can perform poorly on unseen test data, since it treats "never observed in training" as "truly impossible").
- **k $\to \infty$ (very large k)**: washes out the actual observed counts entirely, driving every estimated probability toward a **uniform distribution** over possible values (or toward whatever the weighted prior favors — see below), regardless of the real data.
- **Effect on likelihood of training data**: unsmoothed MLE, by definition, **maximizes** the likelihood of the training data it was fit on — so adding *any* amount of smoothing (k > 0) can only keep the training-data likelihood the same or (generally) **decrease** it, since you're moving away from the values that were exactly tuned to maximize that specific dataset's likelihood.
- **Weighted/asymmetric smoothing**: to encode a *prior belief* that isn't uniform (e.g., "class A is generally twice as likely as class B"), use per-class weights multiplying the smoothing term (not the counts) — e.g., $\dfrac{\text{count} + w_A \cdot k}{N + \sum_i w_i \cdot k}$ with $w_A = 2 \cdot w_B$. Multiplying the smoothing constant (not the raw counts) is what lets the prior bias show up even with zero observed counts; using a larger k expresses higher confidence in that prior (harder for data to overturn it), while a small k allows the data to dominate quickly.

Choosing hyperparameters (e.g., smoothing strength k, or k in k-NN-style models):

- Choosing from the **training set** $\to$ leads to overfitting (will always favor whatever fits training data best, often k=0 or minimal regularization).
- Choosing from the **test set** $\to$ invalid, this is "cheating" (test set must stay unseen until final evaluation).
- Choosing from a separate **validation set** $\to$ correct approach.

Naive Bayes vs. Perceptron for continuous features: Naive Bayes generally struggles with continuous-valued features because its CPTs (as taught) are discrete tables — infinitely many possible continuous values makes exact CPTs intractable. Perceptron handles continuous features natively (it's just a dot product), so for continuous features perceptron/other discriminative models are typically preferred over basic (tabular) Naive Bayes.

13. Decision Trees

Structure: internal nodes test an attribute (splitting on its value), leaves give a predicted label. **You never test the same attribute more than once along a single root-to-leaf path** (once split on, that attribute's value is already fully "used up" for that path).

Choosing the split attribute: pick the attribute that maximizes **information gain** = (entropy before split) $-$ (weighted average entropy after split).

- **Entropy**: $H = \sum_i -p_i \log(p_i)$ over the class distribution at a node (or leaf/branch) — measures impurity/uncertainty. $H = 0$ when a branch is perfectly pure (all one class); higher entropy = more mixed.
- To compute the entropy of a split with multiple branches: compute the entropy within each branch separately, then take a **weighted average** across branches (weighted by the fraction of data points falling into each branch) — do not average the branch entropies unweighted, and do not compute entropy over the un-split full set and call that "the split's entropy."

Multiple valid trees: different trees splitting in a different **order** on the same underlying set of attributes can still represent the exact same function, i.e. **logically equivalent** — $T_1(x) = T_2(x)$ for all x — as long as they agree on every possible input, regardless of which attribute is tested first.

Convergence: with infinite training data, decision tree learning is guaranteed to converge to a tree logically equivalent to the "true" underlying tree (assuming one exists that the data was generated from).

Overfitting & Regularization:

- Deeper/larger trees fit training data increasingly well (never worse), but risk overfitting — memorizing noise instead of learning generalizable structure.
- **Limiting max depth** is a direct regularization technique — trades some training accuracy for better expected generalization to unseen data.
- **pchance**: an estimate of the probability that an observed split's apparent improvement in accuracy is due to random chance rather than a real underlying pattern. **MaxPchance** is a user-set threshold — if a candidate split's pchance exceeds MaxPchance, the split is rejected (pruned) as not statistically justified. MaxPchance is a regularization/tuning **parameter you set**, whereas pchance, entropy, and information gain are all **computed** quantities from the data, not parameters you choose.
- Comparing a depth-limited tree A to an unlimited tree B on the same data: A can have equal or **lower** training accuracy than B (never higher — more splits can only help or not hurt training fit). But A's **validation/test accuracy could be strictly higher** than B's if B overfits. Both trees *could* end up with the same number of leaves in edge cases (e.g., if all data at a node already has identical feature values, adding depth doesn't create new splits).

Decision boundary shape: decision trees with axis-aligned single-feature splits produce **piecewise-linear** (axis-aligned box-like) decision boundaries — can represent nonlinear-looking boundaries overall by combining many linear splits.

Minimum tree depth for perfect classification: relates to how many straight (axis-aligned) cuts are needed to fully separate the classes in feature space — think of it geometrically (how many horizontal/vertical lines are needed to isolate all same-label regions from each other).

14. Perceptron & Linear Classifiers

Multiclass perceptron update: compute the dot product of the weight vector for each class with the feature vector: $W_{\text{class}} \cdot f(x)$, for every class. Predict $\hat{y} = \arg\max_{\text{class}}(W_{\text{class}} \cdot f(x))$.

- If the prediction is **wrong** ($\hat{y} \neq y_{\text{actual}}$): update **only** the two involved weight vectors —
  - $W_{\text{actual}} \leftarrow W_{\text{actual}} + f(x)$ (push the correct class's score up)
  - $W_{\text{predicted}} \leftarrow W_{\text{predicted}} - f(x)$ (push the wrongly-predicted class's score down)
  - All other classes' weights are **unchanged**.
- Repeat over the dataset (potentially multiple passes) until convergence (predictions match actual for all points), if convergence is possible.

Binary perceptron update (equivalent special case): if misclassified, $w \leftarrow w + f(x)$ if the true label is +1 (or $w \leftarrow w - f(x)$ if true label is -1); update direction is $\text{learning rate} \times \text{error} \times \text{feature}$, where $\text{error} = y_{\text{actual}} - y_{\text{predicted}}$ and the weight moves in the direction that reduces future error on this point.

Convergence guarantee: if the data is **linearly separable** (a straight line/hyperplane can perfectly separate the classes), the perceptron algorithm is **guaranteed to converge to zero training error in a finite number of steps**, regardless of initial weights. If the data is **not** linearly separable, the perceptron algorithm is **never** guaranteed to converge — it may oscillate forever, regardless of the initial weight vector or how long you run it.

Making non-separable data separable via feature engineering: adding extra features (even nonlinear transformations of existing ones, e.g., x_1^2, $|x_1 - x_2|$, $x_1 + x_2$, distance-from-a-point features) can make data linearly separable in the new feature space even if it wasn't in the original space. General principle: adding features to an *already*-separable representation **never destroys** separability (you can always ignore the new feature by giving it weight 0) — but adding features to a **non-separable** representation only helps if the specific new feature actually captures the true structure separating the classes (not every added feature works — you need one whose value differs systematically between classes in a way a hyperplane can exploit, e.g., distance to a center point turning a circular boundary into a linear threshold).

15. Neural Networks & Backpropagation

Forward pass (single neuron/layer): $\text{output} = \sigma\left(\sum(\text{weight} \times \text{input}) + \text{bias}\right)$, where σ is an activation function (sigmoid, ReLU, etc.), applied elementwise per neuron.

- To compute a hidden layer's activations: for each hidden unit, sum (incoming weight $\times$ value on that incoming edge) + bias, then pass that sum through the activation function.
- Chain layers: the outputs (post-activation) of one layer become the inputs to the next layer's weighted sums.

Sigmoid function behavior: outputs values in (0,1). If the pre-activation sum z is very negative, $\sigma(z)$ approaches 0; if very positive, approaches 1. A negative pre-activation value pushed through sigmoid rounds down toward 0 in practice for classification purposes.

ReLU: $\text{ReLU}(z) = \max(0, z)$ — outputs 0 for any negative input, passes positive input through unchanged. Not a loss function — it's an activation function (common confusion point: **ReLU, softmax, and sigmoid are activation functions, not loss functions**).

Common loss functions:

- **Regression** (continuous output): Mean Squared Error (MSE), Mean Absolute Error (MAE).
- **Classification** (discrete class output): **Cross-Entropy** is the standard choice (not MSE/MAE — those are for regression).

Parameter counting: for a fully-connected layer with m input units and n output units (ignoring bias), the number of weights = $m \times n$ (every input connects to every output). Total parameters in a network = sum of $m \times n$ across all layers (plus bias terms if included — read the problem carefully on whether to include them).

Backpropagation (chain rule pattern): to compute how the loss changes with respect to an early-layer weight (e.g., $\partial \text{Loss} / \partial w1$), apply the **chain rule** through every intermediate node between the loss and that weight, layer by layer, moving *backward* from the output: $\frac{\partial \text{Loss}}{\partial w1} = \frac{\partial \text{Loss}}{\partial \text{output}} \cdot \frac{\partial \text{output}}{\partial [\text{previous node}]} \cdots \frac{\partial [\text{intermediate}]}{\partial w1}$ Every edge/operation in the computation graph on the path from the weight to the loss contributes one multiplicative factor to this product — trace the path and multiply each local derivative along it.

Gradient descent update rule: $w \leftarrow w - \alpha \cdot \frac{\partial \text{Loss}}{\partial w}$ — **subtract** the gradient (scaled by learning rate α) to move toward lower loss (minimize). (Gradient **ascent**, by contrast, **adds** the gradient to move toward higher values — used when *maximizing* an objective, e.g., likelihood.)

Gradient vector properties: the gradient always points in the direction of **steepest ascent** of the function (perpendicular to the local contour line/level curve). To minimize, move in the *opposite* direction of the gradient (steepest descent).

Local optima: gradient ascent/descent is **not guaranteed to reach the global optimum** when the objective function has multiple local maxima/minima — it can get stuck at a local optimum depending on starting point and step size.

Decision boundary of logistic regression: **always linear** in the original feature space (the sigmoid is applied to a linear combination of inputs, so the boundary where output = 0.5 is exactly where the linear combination = 0) — even though the *output* is a smooth nonlinear curve (probability), the boundary itself is a straight hyperplane. (Some data configurations simply won't have any linear boundary that separates classes well — but whatever boundary logistic regression finds is linear.)

16. Attention & Transformers / LLM Basics

Self-attention mechanism: for each token, compute a **Query** vector, and compare it against every token's **Key** vector via **dot product** — $\text{score} = Q \cdot K$. Higher dot product = token attends more strongly to that key. The token with the highest dot product with a given query is what that query "attends to most."

Softmax over attention scores: raw dot-product scores are passed through softmax to get attention weights (summing to 1), which then weight the **Value** vectors to produce the attention output.

Softmax invariances (useful for reasoning about transformations of scores):

- **Adding the same constant to every score** does **not** change the softmax output (shift-invariant) — $\text{softmax}(z + c) = \text{softmax}(z)$ for a constant c added to all entries.
- **Multiplying every score by the same constant DOES change** the softmax output (not scale-invariant) — this changes how "peaked" or "flat" the resulting distribution is (this is effectively what a "temperature" parameter does).

Advantages of transformers/attention over n-gram (e.g., trigram) models:

- Attention allows **distant-word correlations** to be captured directly (a token can attend to any other token regardless of distance), unlike fixed-window n-gram models which only see a few adjacent words.
- Attention does **not** inherently give more frequent training words higher probability by design, nor does capturing distant correlations by itself let a model assign nonzero probability to entirely unseen sentence structures — that capability comes from the model's learned generalization broadly (via embeddings/subword tokens), not attention's distant-correlation property specifically. Be careful to match the claimed advantage to the actual mechanism when a question lists several options.

Order of pipeline steps: **tokenization happens BEFORE embedding**, not after — raw text is first broken into tokens (subword units), and only then are those discrete tokens mapped to continuous embedding vectors.

Pretraining vs. Fine-tuning:

- **Pretraining** (self-supervised, e.g., next-token prediction on a large text corpus) teaches a model general language patterns — high accuracy on next-token prediction for in-domain text does **not** by itself mean the model is good at being a helpful, instruction-following *assistant* (a pretrained model just continues text in a similar style, it doesn't necessarily know how to respond helpfully to a request/question format if that pattern rarely appeared in training text).
- **Instruction tuning** (fine-tuning on curated instruction-response / helpful-conversation examples) is what teaches a model to actually follow instructions and hold a helpful dialogue — this step uses **supervised learning** (labeled instruction $\rightarrow$ response pairs), not unsupervised learning.
- **RLHF (Reinforcement Learning from Human Feedback)**: further tunes a model using human preference data and a **policy-gradient-style RL method** (not Q-learning) to align outputs with human preferences.

17. Dynamic Decision Networks / Belief-State MDPs (brief)

A **dynamic decision network** extends a dynamic Bayes Net (states evolving over time) with action and reward nodes repeated at each time step, used to model sequential decision-making under partial observability.

- Belief update at each step combines a **time-elapse** component (using the transition model and the previous action) and an **observation** component (using the sensor model) — structurally analogous to HMM belief updates (Section 10), but now conditioned also on the action taken.
- If a new domain requires an action to influence the **reward** directly (not just the state), add an edge from the action node into the reward node for that time step.
- If an action is meant to **reveal information** about the current state (like a "measurement" or "observe" action), that action should influence the **observation** node for that (or the next) time step — not the state node itself (the action doesn't change the true state, only what's revealed about it).
- The belief state at time t is a sufficient statistic for choosing the optimal action at time t — see VPI section above for the reasoning (Markov property at the belief level).
- Particle filtering (Section 10) can be adapted to estimate the belief state in these networks the same way, with the observation-weighting step now also conditioned on the action taken.